JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

E-Learning Material

Computer Architecture

(25VBT0C104)

www.onlinejain.com

Module: 02

Machine Instructions, I/O Systems and Memory Organization

Module Information

Field	Details
Course Title	Computer Architecture
Course Code	25VBT0C104
Programme / Semester	BCA — Semester I
Course Type	Core Course
Module Number	Module 2
Module Title	Machine Instructions, I/O Systems, and Memory Organization
Learning Hours	24 Hours

Module Overview

Building on the foundational understanding of computer structure developed in Module 1, this module takes you deeper into the operational world of how a computer actually communicates with data, devices, and the outside world. Three major themes run through this module: how the CPU accesses its data (addressing modes), how it talks to hardware devices (I/O systems), and how it organises and retrieves information stored in memory (memory organization and hierarchy).

The module begins with addressing modes — the various ways a machine instruction can specify where its operands are located. This is one of the most practically important topics in the course, because every instruction you write in assembly language implicitly uses an addressing mode. You will study eight major addressing modes, from the simplest (immediate mode, where the data is embedded in the instruction itself) to the most flexible (indirect and indexed modes).

Assembly language and instruction encoding form the bridge between the human-readable instructions you write and the binary machine code the processor executes. You will study how instructions are structured as binary fields, how opcodes and operand addresses are packed together, and how a disassembler can reconstruct instructions from binary. Subroutines — the building blocks of all structured programming — are examined in detail, including how the stack enables nested subroutine calls and parameter passing.

The I/O section of the module addresses one of the most complex aspects of computer architecture: how the CPU communicates with the enormous variety of devices attached to a computer system. You will study the three fundamental I/O techniques (programmed I/O, interrupt-driven I/O, and DMA), understand how interrupt controllers work, and examine the leading modern I/O interface standards: PCIe 6.0 (released 2022), USB4 Generation 3x2 (2022), NVMe over PCIe, and SAS/SCSI as used in enterprise storage systems.

The module closes with memory organisation — a topic that directly affects every program's performance. You will understand the detailed structure of RAM and ROM types available in 2024–2025 systems (DDR5, LPDDR5X, NAND Flash), and develop a deep understanding of the memory hierarchy — the principle that a well-designed computer uses a cascade of memory types, from ultra-fast but tiny registers down to vast but slower mass storage, to achieve the best balance of speed, capacity, and cost.

Learning Objectives

By the end of this module, you will be able to:

- Identify and explain all major addressing modes and apply them to determine effective addresses from given instructions. (BTL 2 & 3)
- Explain how assembly language instructions are encoded into binary machine code, and decode a given binary instruction. (BTL 2)
- Describe how subroutines are called and returned from, and trace the state of the stack during nested subroutine calls. (BTL 2 & 3)
- Compare the three I/O techniques (programmed I/O, interrupt-driven I/O, and DMA), and explain when each is appropriate. (BTL 2 & 3)
- Explain the memory hierarchy principle and describe the characteristics of each level from registers to secondary storage. (BTL 2)

Learning Outcomes

Upon successful completion of this module, you will be able to:

- Compute the effective address of an operand for any addressing mode when given an instruction, register contents, and memory values.
- Decode a given binary machine instruction into its assembly language equivalent by identifying opcode and operand fields.
- Trace the push/pop operations on the stack during a two-level nested subroutine call, showing all register and stack-pointer values.
- Distinguish between programmed I/O, interrupt-driven I/O, and DMA, and justify the choice of technique for a given device and application.
- Construct a memory hierarchy diagram with correct values for access time, capacity, and cost per bit for each level in a modern 2024–25 system.

Table of Topics

Unit	Topic / Sub-Unit	Key Concepts Covered
2.1	Addressing Modes	Immediate, register, direct, indirect, register-indirect, indexed, relative, auto-increment/decrement
2.2	Assembly Language and Instruction Encoding	Assembly mnemonics, opcode fields, operand encoding, fixed-length vs. variable-length encoding
2.3	Subroutines and Procedure Calls	CALL, RETURN, stack frame, activation record, parameter passing, nested calls
2.4	Input and Output Operations	Programmed I/O (polling), memory-mapped vs. port-mapped I/O, I/O registers
2.5	Interrupt Mechanism and Hardware	Interrupt request, ISR, interrupt vector table, APIC, nested interrupts, maskable vs. non-maskable
2.6	Direct Memory Access (DMA)	DMA controller, DMA transfer cycle, burst mode, cycle-stealing, scatter-gather, IOMMU
2.7	Standard I/O Interfaces: PCIe, NVMe, USB, SAS/SCSI	PCIe 6.0/7.0, USB4 Gen 3x2, NVMe 2.0, SAS-4, interface speeds and real-world applications
2.8	Memory Organization: RAM and ROM	SRAM, DRAM, DDR5, LPDDR5X, SDRAM timing, EEPROM, Flash NAND/NOR, 3D NAND, QLC
2.9	Memory Hierarchy and Speed-Cost Trade-offs	Locality of reference, hierarchy levels, access time, bandwidth, cost/bit, working set principle

2.1 Addressing Modes

When the CPU executes an instruction, it needs to know where to find the data (operands) it will operate on. Should it look inside the instruction itself? Inside a register? At a specific memory address? The answer depends on the addressing mode specified in the instruction. An addressing mode is the method by which a machine instruction identifies or computes the location of its operand.

Addressing modes are one of the most powerful and flexible features of an instruction set architecture. Different addressing modes are optimised for different use cases: accessing constants, traversing arrays, dereferencing pointers, implementing relative jumps, and implementing stacks. A rich set of addressing modes can reduce code size and improve performance — but adds complexity to the CPU's instruction decoder. Understanding addressing modes thoroughly is essential for anyone who writes assembly code or studies compiler design.

Key Concept

The EFFECTIVE ADDRESS (EA) is the actual memory address where the operand is located.

Different addressing modes compute the EA differently:

— Some use the address embedded directly in the instruction.

— Some add a register value to an offset.

— Some dereference a pointer stored in a register or memory.

The CPU computes the EA during the decode/execute phase using the specified mode.

1. Immediate Addressing Mode

In immediate addressing, the operand is not an address at all — the actual data value is embedded directly within the instruction itself. The data is immediately available as part of the instruction; no additional memory access is needed to fetch the operand.

- Syntax (ARM-style): MOV R1, #25 (load the value 25 directly into register R1)
- Effective Address: Not applicable — the operand IS the data. No memory lookup needed.
- Use Case: Loading constants, small fixed values, initialising loop counters.
- Advantage: Very fast — operand is available as soon as the instruction is decoded.
- Limitation: The operand is fixed at compile time; the range of values is limited by the number of bits available for the immediate field.

2. Register Addressing Mode

In register addressing, the operand is located in one of the CPU's internal registers. The instruction specifies the register number, and the CPU reads the operand directly from that register — no memory access required.

- Syntax: ADD R1, R2 (add the value in register R2 to register R1)
- Effective Address: Not applicable — operand is in a register, not memory.
- Use Case: All ALU operations on data that is already loaded into registers.
- Advantage: Fastest addressing mode for operand access — register access is much faster than memory access.

3. Direct (Absolute) Addressing Mode

In direct addressing, the instruction contains the actual memory address of the operand. The CPU places this address on the address bus and reads the operand from that memory location. This is a single-level memory access.

- Syntax: LOAD R1, 2050 (load the value stored at memory address 2050 into R1)
- Effective Address: EA = Address field of the instruction. EA = 2050.
- Use Case: Accessing global variables, fixed data tables, fixed I/O port addresses.
- Limitation: The address is fixed at assembly time. Cannot be used to access dynamically computed locations.

4. Indirect Addressing Mode

In indirect addressing, the instruction contains a memory address, but that address does not hold the operand — it holds the address of the operand. The CPU must perform two memory accesses: first to fetch the pointer (the address of the operand), and then to fetch the operand itself. This is sometimes called double indirection.

- Syntax: LOAD R1, (2050) (parentheses indicate indirection)
- Effective Address: EA = Memory[address in instruction] = Memory[2050].
- Use Case: Pointer dereferencing. If memory location 2050 contains the value 3000, the operand is at address 3000.
- Disadvantage: Requires two memory accesses — slower than direct addressing.

5. Register Indirect Addressing Mode

Register indirect addressing is similar to indirect addressing, but the pointer (the address of the operand) is held in a register rather than in memory. This is extremely common in modern processors because registers are much faster to access than memory, and this mode enables efficient pointer manipulation.

- Syntax: LOAD R1, (R2) (R2 holds the address; load from that address into R1)
- Effective Address: EA = contents of register R2. If R2 = 3000, then EA = 3000.
- Use Case: Pointer dereferencing in C (the * operator compiles to register indirect addressing).
- Example: In C, *ptr = 42; — if ptr is stored in R2, this compiles to STORE (R2), #42.

6. Indexed Addressing Mode

In indexed addressing, the effective address is computed by adding the contents of an index register to a constant base address (or displacement) specified in the instruction. This mode is ideal for traversing arrays, because incrementing the index register moves the access point through consecutive array elements.

- Syntax: LOAD R1, 1000(R3) (EA = 1000 + contents of R3)
- Effective Address: EA = Base address (from instruction) + Index register value.
- Use Case: Array access. If array A starts at address 1000 and each element is 4 bytes, then element A[i] is at address 1000 + 4*i. Store 4*i in R3, and access A[i] with the instruction LOAD R1, 1000(R3).

7. Base-Register (Base + Displacement) Addressing Mode

In base-register addressing, the effective address is computed by adding a displacement (offset) in the instruction to the contents of a base register. This is similar to indexed addressing, but semantically the base register holds the starting address of a data structure (such as a record or stack frame), and the displacement selects a specific field within it.

- Syntax: LOAD R1, 12(R5) (EA = R5 + 12)
- Effective Address: EA = Base register + Displacement.
- Use Case: Accessing fields of a record/struct; accessing local variables on the stack frame (R5 = frame pointer, displacement = offset of the variable in the frame).

8. Relative Addressing Mode

In relative addressing, the effective address is computed by adding a signed displacement (offset) in the instruction to the current value of the Program Counter (PC). Since the PC always points to the next instruction, this produces a PC-relative address — one that is expressed as a distance from the current instruction. This mode is almost universally used for branch (jump) instructions.

- Syntax: BRANCH +12 (jump to the address PC + 12)
- Effective Address: EA = PC + Displacement. If PC = 2000 and displacement = +12, then EA = 2012.
- Use Case: Conditional and unconditional branches in all modern architectures.
- Key Advantage: Position-independent code — programs can be loaded at any memory address and the relative branches still work correctly, because they are relative to the PC, not absolute.

Mode	EA Calculation	Example	Primary Use
Immediate	*Operand = value in instr.*	*MOV R1, #25*	Load constants
Register	*Operand = Reg[r]*	*ADD R1, R2*	ALU operations on regs
Direct	*EA = Addr in instruction*	*LOAD R1, 2050*	Global/fixed variables
Indirect	*EA = Mem[Addr in instr]*	*LOAD R1, (2050)*	Pointer dereferencing (slow)
Register Indirect	*EA = Reg[r]*	*LOAD R1, (R2)*	Pointer dereferencing (fast)
Indexed	*EA = Base + Index Reg*	*LOAD R1, 1000(R3)*	Array element access
Base+Displacement	*EA = Base Reg + Disp*	*LOAD R1, 12(R5)*	Struct fields, stack frames
Relative (PC)	*EA = PC + Displacement*	*BRANCH +12*	Branch/jump instructions

Worked Example: Addressing Mode — Effective Address Calculation

Given: Instruction = LOAD R1, 500(R4)

Register R4 = 2000

Memory[2500] = 0xFF

Mode: Indexed Addressing

EA = Base (500) + Index Register (R4 = 2000) = 2500

Operand = Memory[2500] = 0xFF

Result: R1 ← 0xFF

Now if R4 is incremented to 2004:

EA = 500 + 2004 = 2504 → accesses the next array element automatically.

This is why indexed addressing is ideal for array traversal.

2.2 Assembly Language and Instruction Encoding

Assembly language is the human-readable representation of a computer's machine language. Every processor has its own Instruction Set Architecture (ISA) — a defined set of binary-encoded machine instructions it can execute. Assembly language provides a symbolic, mnemonic form of these instructions, making them readable and writable by humans. An assembler program translates assembly language source code into binary machine code.

Understanding instruction encoding — how instructions are converted into binary — is crucial for understanding how the CPU decodes instructions in the Decode phase of the fetch-decode-execute cycle, and for understanding performance characteristics such as instruction size, code density, and pipeline behaviour.

Levels of Language

Computer programs can be written at different levels of abstraction:

- High-Level Language (HLL): C, Java, Python — human-friendly, portable, but far from machine code.
- Assembly Language: Symbolic representation of machine instructions. Architecture-specific. Direct control over hardware.
- Machine Language: Binary (0s and 1s) directly executed by the CPU.

The path from C to machine code: C source → Compiler → Assembly code → Assembler → Object code → Linker → Executable (machine code).

Basic Assembly Language Instructions

Assembly instructions are written as: MNEMONIC Destination, Source(s)

Common instruction categories found in most ISAs:

Category	Example (x86-64 style)	Meaning
Data Transfer	*MOV RAX, 100*	Load 100 into register RAX
Data Transfer	*MOV [2050], RAX*	Store value of RAX at memory address 2050
Arithmetic	*ADD RAX, RBX*	RAX = RAX + RBX
Arithmetic	*SUB RCX, 5*	RCX = RCX - 5
Arithmetic	*MUL RBX*	RDX:RAX = RAX * RBX (64-bit multiply)
Logic	*AND RAX, RBX*	RAX = RAX AND RBX (bitwise)
Logic	*OR RAX, RBX*	RAX = RAX OR RBX
Logic	*SHL RAX, 2*	Shift RAX left by 2 (multiply by 4)
Control Flow	*JMP 3000*	Unconditional jump to address 3000
Control Flow	*JZ +8*	Jump forward 8 bytes if Zero flag = 1
Control Flow	*CALL FUNC_NAME*	Call subroutine (push return address, jump)
Control Flow	*RET*	Return from subroutine (pop return address, jump)
Stack	*PUSH RAX*	Decrement SP; store RAX at address [SP]
Stack	*POP RBX*	Load value at [SP] into RBX; increment SP

Instruction Encoding: Fixed-Length vs. Variable-Length

The way instructions are encoded into binary has a profound impact on processor design and performance. There are two primary approaches:

Fixed-Length Encoding (RISC approach)

Every instruction is exactly the same number of bits — typically 32 bits in RISC architectures (ARM, MIPS, RISC-V). This simplifies the decoder hardware enormously: the processor always knows where one instruction ends and the next begins, enabling straightforward pipelining and instruction prefetching.

- Example: ARM A64 (AArch64) — every instruction is exactly 32 bits. Used in Apple M-series (M3 Pro/Max, 2023), Qualcomm Snapdragon 8 Gen 3 (2023), and all modern Android flagship SoCs.
- Example: RISC-V — open-source ISA with 32-bit fixed-length base instructions. Used in India's indigenous processor development (SHAKTI, IIT Madras, 2024) and SiFive's commercial RISC-V chips.

Variable-Length Encoding (CISC approach)

Instructions can be different sizes — from 1 byte to 15 bytes in the Intel x86-64 architecture. This allows a richer, more compact instruction set that can encode complex operations, but complicates the decoder because the processor must identify the boundary between instructions dynamically.

- Example: Intel x86-64 — variable-length instructions from 1 to 15 bytes. Used in Intel Core Ultra 200 series (Arrow Lake, released Q4 2024) and AMD Ryzen 9000 series (Zen 5 architecture, 2024).

Worked Example: Binary Instruction Encoding — RISC-V 32-bit R-type Format

A RISC-V R-type instruction (e.g., ADD rd, rs1, rs2) is encoded as:

Bits [31:25]	Bits [24:20]	Bits [19:15]	Bits [14:12]	Bits [11:7]	Bits [6:0]
funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Total = 7+5+5+3+5+7 = 32 bits

For the instruction: ADD R3, R1, R2 (rd=R3, rs1=R1, rs2=R2)

- funct7 = 0000000 (identifies ADD vs. SUB — same opcode, different funct7)
- rs2 = 00010 (R2 = register 2)
- rs1 = 00001 (R1 = register 1)
- funct3 = 000 (ADD function code)
- rd = 00011 (R3 = destination register 3)
- opcode = 0110011 (R-type ALU operation)

Binary: 0000000 00010 00001 000 00011 0110011

Hex: 0x00208033

2.3 Subroutines and Procedure Calls

A subroutine (also called a procedure or function) is a named, reusable block of instructions that performs a specific task. Subroutines are the fundamental unit of code organisation in all programming paradigms: every function you write in Python, Java, or C compiles to a subroutine at the machine level. Understanding how subroutines work at the hardware level — how they are called, how they receive parameters, and how they return values — is essential for understanding program execution, stack overflows, function call overhead, and recursion.

The CALL and RETURN Mechanism

When the CPU encounters a CALL instruction (the machine-level equivalent of a function call), it must:

- Step 1 — Save the return address: The address of the instruction immediately after the CALL (the return address) must be saved, so the CPU can come back to the right place after the subroutine finishes.
- Step 2 — Jump to the subroutine: The PC is set to the starting address of the subroutine.
- Step 3 (at end of subroutine) — RETURN: Retrieve the saved return address and load it into the PC, resuming execution after the original CALL instruction.

Early processors saved the return address in a single register — which meant subroutines could not call other subroutines (re-entrant/nested calls were impossible). The solution is the stack.

The Stack and Stack Pointer

The stack is a region of memory organised as a Last-In, First-Out (LIFO) data structure. It is managed by the Stack Pointer (SP) register, which always points to the top of the stack (the most recently pushed item). When something is pushed onto the stack, the SP is decremented (stacks usually grow downward in memory) and the value is written at the new SP address. When something is popped, the value is read from [SP] and SP is incremented.

Key Concept

The stack solves the nested subroutine problem:

CALL: PUSH return address onto stack, then JUMP to subroutine.

RETURN: POP return address from stack, then JUMP to that address.

Because the stack is LIFO, nested calls are handled correctly:

Main calls A → A's return address pushed.

A calls B → B's return address pushed on top.

B returns: pops B's return address (correct).

A returns: pops A's return address (correct).

The stack unwinds in the correct order, regardless of nesting depth.

Stack Frame (Activation Record)

Each subroutine call creates a stack frame (also called an activation record) — a region of the stack allocated for that call. The stack frame contains: the return address, the values of registers saved before the call, the subroutine's local variables, and the parameters passed to the subroutine. The Frame Pointer (FP) or Base Pointer (BP)

register holds the base address of the current stack frame, enabling stable access to local variables even as SP changes.

Worked Example: Stack State During Nested Subroutine Calls

Scenario: main() calls functionA(), which calls functionB().

INITIAL STATE (before any calls):

SP → [top of stack, empty]

AFTER main() calls functionA():

```
SP → | Return address to main |  ← top of stack
     | Saved registers        |
     | Local variables of A   |
```

AFTER functionA() calls functionB():

```
SP → | Return address to A    |  ← top of stack
     | Saved registers        |
     | Local variables of B   |
     |------------------------|
     | Return address to main |
     | Saved registers        |
     | Local variables of A   |
```

functionB() executes RETURN:

→ Pops "Return address to A" from top of stack

→ SP moves down to A's frame, PC = Return address to A

→ functionA() resumes at the instruction after its call to B.

functionA() executes RETURN:

→ Pops "Return address to main" from stack

→ PC = Return address to main

→ main() resumes where it called A.

2.4 Input and Output Operations

Input/Output (I/O) operations are how the CPU communicates with devices: keyboards, displays, storage drives, network interfaces, sensors, and printers. I/O is one of the most architecturally complex aspects of a computer system, because the devices attached to a computer are enormously varied — some are extremely fast (NVMe SSDs can transfer at 14 GB/s), while others are very slow (a keyboard generates an interrupt a few times per second). Managing this diversity efficiently requires well-designed I/O mechanisms.

How the CPU Communicates with I/O Devices

Every I/O device is controlled through one or more I/O registers — small registers inside the device's controller circuit that the CPU can read from and write to. These registers serve specific functions:

- Status Register: The device writes its current state here (ready, busy, error, data available). The CPU reads this to check if the device is ready.
- Control Register: The CPU writes command codes here to instruct the device what to do (start transfer, reset, set mode).

- Data Register: The CPU reads data received from the device, or writes data to be sent to the device.

Two Schemes for Locating I/O Registers

1. Port-Mapped I/O (Isolated I/O)

I/O registers are accessed through a separate address space, distinct from the memory address space. The processor has dedicated I/O instructions (IN and OUT in x86) that read/write to I/O port addresses. This was the approach in early Intel processors and is still supported in x86 architecture for backward compatibility.

- Example (x86): IN AL, 0x60 — reads a byte from keyboard I/O port 0x60 into register AL.

2. Memory-Mapped I/O (MMIO)

I/O registers are mapped into the regular memory address space — each register is assigned a specific memory address. The CPU accesses them using normal LOAD and STORE instructions. This is the dominant approach in all modern systems, including ARM, RISC-V, and modern x86 systems for most devices. Memory-mapped I/O is simpler, more flexible, and enables the use of the full addressing capability and addressing modes for I/O access.

- Example: In an ARM microcontroller (e.g., STM32), the GPIO output register for port A might be at address 0x40020014. Writing a value to this address turns GPIO pins on or off.
- Example: The framebuffer of a GPU in a PC is memory-mapped — the CPU writes pixel values to specific memory addresses to update the display.

Programmed I/O (Polling)

In programmed I/O (also called polling), the CPU actively waits for an I/O device to complete its operation. The CPU continuously reads the device's status register in a tight loop until the device indicates it is ready. This is the simplest I/O technique to implement.

Worked Example: Polling Loop (Programmed I/O)

Wait for keyboard key press (polling):

```
POLL_LOOP:
    LOAD  R1, STATUS_REG    ; Read keyboard status register
    AND   R1, #01h          ; Test bit 0 (data-ready flag)
    JZ    POLL_LOOP         ; If bit = 0, no data yet — keep polling
    LOAD  R2, DATA_REG      ; Bit = 1, key pressed — read the key code
    ; Process key code in R2...
```

Problem: While polling, the CPU is 100% occupied and cannot do other work. This is acceptable for very simple embedded systems but wastes CPU cycles in multi-tasking environments where other processes need the processor.

🔍 Analysing Misconceptions: Polling vs. Interrupts

❌ MYTH: Polling and interrupts both serve the same purpose — detecting when a device needs attention — so the choice between them is simply a matter of programming style.

✅ REALITY: Polling and interrupts have fundamentally different performance characteristics. Polling wastes CPU cycles in a busy-wait loop, consuming 100% of CPU time even when no I/O event has occurred. It is only appropriate when the CPU has nothing else to do or when I/O events occur very frequently. Interrupt-driven I/O allows the CPU to execute other useful work and only respond to a device when the device itself signals that it is ready, via a hardware interrupt signal. The choice is an architectural decision that affects system throughput, latency, and power consumption — not merely a style preference.

2.5 Interrupt Mechanism and Hardware

An interrupt is a hardware signal sent by a device (or software event) to the CPU, requesting immediate attention. Interrupts solve the fundamental problem of programmed I/O: they allow the CPU to continue executing its main program and only respond to a device when that device needs service. This makes efficient multitasking possible and is the basis of all modern operating system event handling.

Think of an interrupt as a doorbell at your home. You do not stand at the door continuously checking if someone is there (that would be polling). Instead, you go about your day (execute your main program), and when the doorbell rings (interrupt occurs), you stop what you are doing, answer the door (execute the Interrupt Service Routine), and then return to what you were doing (resume main program). The key is

that the doorbell can ring at any time — and your work is only interrupted when it does.

The Interrupt Process — Step by Step

- Step 1 — Device signals interrupt: The I/O device raises its Interrupt Request (IRQ) line when it needs CPU attention (e.g., keyboard has data ready, disk transfer complete, timer expired).
- Step 2 — CPU acknowledges: The CPU completes its current instruction, then checks for pending interrupts. If an interrupt is pending and interrupts are enabled (not masked), the CPU asserts an Interrupt Acknowledge (INTA) signal.
- Step 3 — Save processor state: The CPU automatically saves the current PC (return address) and key registers (or the entire CPU state, depending on architecture) onto the stack. This preserves the state of the interrupted program.
- Step 4 — Identify the interrupt source: The CPU determines which device caused the interrupt (via an interrupt vector number provided by the device or interrupt controller).
- Step 5 — Jump to ISR: The CPU uses the interrupt vector number to index into the Interrupt Vector Table (IVT) and retrieves the address of the Interrupt Service Routine (ISR) for that device.
- Step 6 — Execute ISR: The ISR runs: reads data from the device, services the request, and clears the interrupt flag in the device.
- Step 7 — RETURN from interrupt: The ISR executes a special RETI (Return from Interrupt) instruction. The CPU restores the saved state from the stack and resumes the interrupted program from exactly where it was paused.

Interrupt Vector Table (IVT)

The Interrupt Vector Table is a data structure in memory that maps interrupt numbers to the starting addresses of their corresponding Interrupt Service Routines. In x86-64 systems, this is called the Interrupt Descriptor Table (IDT). The CPU uses the interrupt number (vector) as an index into the IVT/IDT to find the ISR address.

Modern Interrupt Controllers: APIC

In modern multi-core x86 systems (Intel Core Ultra 200 series, AMD Ryzen 9000 series as of 2024), interrupt routing is managed by the Advanced Programmable Interrupt Controller (APIC) architecture, which has replaced the older 8259A PIC. The APIC system consists of two components:

- Local APIC (LAPIC): One per CPU core — receives interrupts and delivers them to the core. Also handles inter-processor interrupts (IPIs) for multi-core communication and local timer interrupts.
- I/O APIC: One per system (or one per chipset) — receives external interrupt signals from devices and routes them to the appropriate LAPIC on the appropriate core, according to programmable routing tables. Modern systems use MSI (Message Signalled Interrupts) via PCIe, which bypass the I/O APIC entirely.

Types of Interrupts

Type	Trigger	Example
Hardware Interrupt (IRQ)	Signal from external device	Keyboard data ready, disk DMA complete, NIC packet received
Software Interrupt (INT)	Executed by program (INT instruction)	System call to OS (INT 0x80 in Linux x86), debugging breakpoints
Timer Interrupt	Programmable interval timer expires	OS scheduler tick (typically every 1–4 ms); triggers context switch
Exception / Trap	CPU detects error condition	Division by zero, page fault (virtual address not in RAM), illegal instruction
Maskable Interrupt	Can be disabled by setting IF flag = 0	All hardware IRQs; disabled briefly during critical sections
Non-Maskable Interrupt (NMI)	Cannot be disabled	Hardware failure signal, watchdog timeout, power-loss warning

✓ Let's Check 1	
Q.No	**Question**
1	MCQ: In indexed addressing mode with the instruction LOAD R1, 2000(R5), if R5 = 500, what is the effective address? a) 2000 b) 500 c) 2500 d) 1500
2	MCQ: What is the primary advantage of interrupt-driven I/O over programmed I/O (polling)? a) Interrupt-driven I/O is simpler to implement. b) The CPU can execute other work while waiting for the device, instead of busy-waiting. c) Interrupts use less memory than polling loops. d) Interrupt-driven I/O transfers data faster than polling.
3	Fill in the Blank: In relative (PC-relative) addressing mode, the Effective Address is computed as PC + _____.
4	Fill in the Blank: The data structure in memory that maps interrupt numbers to the addresses of their Interrupt Service Routines is called the Interrupt _____ Table.
5	True/False: When a CALL instruction is executed, the CPU pushes the return address onto the stack before jumping to the subroutine.
6	True/False: Memory-mapped I/O uses dedicated IN and OUT instructions that are separate from normal memory access instructions.

Answers	
1	c) 2500. EA = Base (2000) + Index Register R5 (500) = 2500.
2	b) The CPU can execute other work while waiting for the device. Polling wastes CPU cycles in a busy-wait loop; interrupts free the CPU to do useful work and only service the device when it is actually ready.
3	Displacement (Offset). EA = PC + Displacement. Used for all branch/jump instructions in modern architectures.
4	Vector (Interrupt Vector Table / IVT, or Interrupt Descriptor Table in x86-64).
5	True — CALL saves the return address (address of the next instruction after CALL) onto the stack, then loads the PC with the subroutine address. RETURN pops this saved address back into the PC.
6	False — Memory-mapped I/O uses NORMAL LOAD and STORE instructions, the same as memory access. Port-mapped (isolated) I/O uses dedicated IN and OUT instructions. Modern systems predominantly use memory-mapped I/O.

2.6 Direct Memory Access (DMA)

Direct Memory Access (DMA) is a hardware mechanism that allows certain devices to transfer data directly to and from main memory without continuous involvement of the CPU. It is one of the most important performance optimisations in computer architecture, because without DMA, every byte of a large data transfer (such as loading a 1 GB file from an SSD or receiving a network packet) would require the CPU to execute a separate instruction — tying up the processor with simple data movement and preventing it from doing any useful computation.

The DMA Controller

The DMA Controller (DMAC) is a dedicated hardware component that takes over the bus and performs memory transfers independently of the CPU. When a transfer is needed, the CPU programs the DMAC with: the source address (where to read from — device or memory), the destination address (where to write — memory or device), the number of bytes to transfer, and the direction (device-to-memory or memory-to-device). The DMAC then executes the transfer autonomously.

DMA Transfer Steps

- Step 1 — CPU initiates: CPU writes the transfer parameters (source, destination, byte count) into the DMAC's configuration registers.
- Step 2 — CPU releases bus: CPU may continue executing instructions from its cache while the DMAC takes over the memory bus.
- Step 3 — DMAC executes transfer: DMAC requests the bus from the bus arbiter, gains control, and moves data directly between the device and memory — one word at a time or in bursts.

- Step 4 — DMAC signals completion: When the full transfer is complete, the DMAC raises an interrupt line to notify the CPU.
- Step 5 — CPU processes result: The CPU handles the interrupt, processes the data now in memory, and initiates the next operation.

DMA Modes

- Burst Mode: The DMAC takes control of the bus and transfers the entire block of data in one continuous burst without releasing the bus. Fastest mode but blocks CPU memory access for the duration.
- Cycle-Stealing Mode: The DMAC 'steals' one bus cycle at a time from the CPU — transfers one word, releases the bus, and repeats. Slower than burst mode but allows the CPU to continue executing (from cache). Used when CPU access to memory must not be completely blocked.
- Transparent (Background) Mode: The DMAC only uses the bus when the CPU is not using it (e.g., during internal ALU operations). Slowest but completely transparent to the CPU.

Scatter-Gather DMA

Modern DMA systems support scatter-gather DMA (also called vectored DMA), where a single DMA operation can transfer data to or from multiple non-contiguous memory regions. The DMAC is programmed with a scatter-gather list — an array of (address, length) pairs describing multiple memory regions. This is critical for high-performance networking and storage (e.g., NVMe SSDs and 100 Gbps network cards use scatter-gather DMA extensively in 2024–25 systems).

Did You Know?

Modern NVMe SSDs (such as the Samsung 990 Pro or WD Black SN850X as of 2024) can achieve sequential

read speeds of up to 7,450 MB/s over PCIe 4.0 x4, and next-generation PCIe 5.0 NVMe drives

(e.g., Crucial T705, Corsair MP700 Pro SE) reach up to 14,000 MB/s.

At these speeds, every byte transfer using programmed I/O would require approximately 2 billion CPU

instructions per second just to move data — making DMA not a luxury but an absolute necessity.

Without DMA, the CPU would be a full-time data mover with no capacity for actual computation.

IOMMU: Security and Virtualisation for DMA

Modern systems include an Input-Output Memory Management Unit (IOMMU) — a hardware unit that provides memory address translation and access control for DMA operations. The IOMMU prevents a compromised or malicious device from performing a DMA attack — directly reading or writing arbitrary physical memory addresses. Intel's implementation is called Intel VT-d (Virtualisation Technology for Directed I/O); AMD's equivalent is AMD-Vi/IOMMU. Both are standard features in 2024–25 server and desktop processors. The IOMMU is also essential for device assignment in virtualisation environments, enabling virtual machines to use physical devices directly with full performance.

2.7 Standard I/O Interfaces: PCIe, NVMe, USB, and SAS/SCSI

I/O interfaces (also called buses or interconnects) are standardised hardware and protocol specifications that define how devices physically connect to and communicate with the computer system. Standardisation is critical: it means that any PCIe-compliant device works in any PCIe slot, regardless of manufacturer. In this unit, we examine the four most important I/O interface standards in current use as of 2024–2025.

1. PCIe — PCI Express

PCIe (Peripheral Component Interconnect Express) is the dominant high-speed I/O bus standard in modern computers, used for graphics cards (GPUs), NVMe storage controllers, Wi-Fi cards, capture cards, RAID controllers, and virtually all other high-performance expansion devices. PCIe is a serial point-to-point interface — unlike the older PCI (which was a shared parallel bus), each PCIe device has a dedicated link to the CPU or chipset.

A PCIe link is composed of lanes — each lane is a pair of differential signal pairs (one for transmit, one for receive). A device can use x1 (1 lane), x2, x4, x8, or x16 lanes. Bandwidth scales linearly with lane count.

Version	Year	Speed/Lane (each dir)	x16 Bandwidth (each dir)	Key Usage (2024-25)
PCIe 3.0	2010	~1 GB/s	~16 GB/s	Legacy systems; still common in mid-range PCs
PCIe 4.0	2017	~2 GB/s	~32 GB/s	AMD Ryzen 5000, Intel 12th-13th Gen; PCIe 4.0 NVMe SSDs (7 GB/s)
PCIe 5.0	2019	~4 GB/s	~64 GB/s	Intel Core 14th Gen, AMD Ryzen 7000; PCIe 5.0 NVMe SSDs (14 GB/s)
PCIe 6.0	2022	~8 GB/s	~128 GB/s	AI accelerators, HPC; Intel Xeon, AMD EPYC server CPUs (2024)
PCIe 7.0	2025 (spec)	~16 GB/s	~256 GB/s	Finalised spec 2025; targeting AI/ML training clusters, 800G networking

PCIe uses Message-Signalled Interrupts (MSI and MSI-X) — the device writes a message directly to a specific memory address to signal an interrupt, bypassing the legacy I/O APIC. MSI-X supports up to 2,048 interrupt vectors per device and is essential for the multi-queue operation of high-speed NVMe SSDs and 100 Gbps NICs (Network Interface Cards).

2. NVMe — Non-Volatile Memory Express

NVMe (Non-Volatile Memory Express) is a protocol and logical device interface specification designed specifically for flash-based storage (SSDs) connected over PCIe. Before NVMe, SSDs used the AHCI (Advanced Host Controller Interface) protocol originally designed for spinning hard disks — a protocol that supported only a single command queue with a depth of 32. NVMe was designed from scratch for the parallelism of flash storage.

- NVMe Key Features: Up to 65,535 I/O queues, each with up to 65,535 commands — enabling massive parallelism. Dramatically lower latency (20–100 microseconds vs. 1–3 milliseconds for SATA SSDs). Direct PCIe attachment — no SATA or SAS controller needed.
- NVMe 2.0 (released June 2021, widely deployed 2023–25): Introduced NVM Express Base Specification 2.0, standardising NVMe over Fabrics (NVMe-oF) for network-attached NVMe storage. Supports PCIe 5.0 and upcoming PCIe 6.0 host interfaces.
- Real-world 2024–25 NVMe drives: Samsung 990 Pro (PCIe 4.0, 7,450 MB/s read), WD Black SN850X (PCIe 4.0, 7,300 MB/s), Crucial T705 (PCIe 5.0, 14,500 MB/s read — one of the fastest consumer NVMe drives available as of 2024).

3. USB — Universal Serial Bus

USB (Universal Serial Bus) is the universal peripheral interface — used for keyboards, mice, external drives, phones, cameras, audio interfaces, and virtually every low-to-medium speed peripheral. USB is designed for simplicity, automatic device recognition (plug-and-play), and power delivery, making it the dominant standard for consumer device connectivity.

Version	Max Speed	Connector	Key Features / Year
USB 2.0	480 Mb/s (60 MB/s)	Type-A, Type-B, Micro	Still standard for keyboards, mice, low-speed peripherals (2000)
USB 3.2 Gen 1	5 Gb/s	Type-A or Type-C	SuperSpeed USB; flash drives, webcams (2008/2017 name)
USB 3.2 Gen 2	10 Gb/s	Type-A or Type-C	SuperSpeed+ USB; external SSDs at full speed (2013/2017)
USB 3.2 Gen 2x2	20 Gb/s	Type-C only	Two 10 Gb/s lanes; high-speed docks (2017)
USB4 Gen 2x2	20 Gb/s	Type-C only	Based on Thunderbolt 3; PD 3.0 up to 240W (2019)
USB4 Gen 3x2	40 Gb/s	Type-C only	Thunderbolt 4 compatible; current flagship (2021-2025)
USB4 Version 2.0	80 Gb/s	Type-C only	Announced 2022; shipping in devices 2023-24; used in Thunderbolt 5

USB4 Version 2.0 (80 Gb/s, announced late 2022, shipping in Intel Meteor Lake/Core Ultra 200 systems 2024) is compatible with Thunderbolt 5, which also delivers 120 Gb/s asymmetric bandwidth (120 Gb/s download + 40 Gb/s upload) for external GPU and display use cases. USB Power Delivery 3.1 (PD 3.1) supports up to 240 W charging — enabling USB-C laptop charging even for high-end gaming laptops.

4. SAS / SCSI — Enterprise Storage Interfaces

SCSI (Small Computer System Interface) is a long-established set of standards for connecting and transferring data between computers and storage devices, originally developed in the 1980s. In modern enterprise systems, the serial version — SAS (Serial Attached SCSI) — is the dominant interface for enterprise HDDs and SSDs in servers and data centres.

- SAS-3 (12 Gb/s, 2013): Widely deployed in data centre HDDs and enterprise SSDs as of 2024. Standard interface in HPE ProLiant, Dell PowerEdge, and Lenovo ThinkSystem servers.
- SAS-4 (22.5 Gb/s): Standardised by INCITS in 2017; adoption in high-performance enterprise storage arrays (all-flash arrays from Pure Storage, NetApp AFF, and IBM FlashSystem as of 2024).
- SAS vs. SATA vs. NVMe: SAS supports multi-path I/O (redundant paths to storage — critical for high availability), full-duplex operation, and longer cable distances than SATA. NVMe is now faster but SAS remains dominant in enterprise storage due to its reliability, dual-path capability, and ecosystem maturity.
- Future: NVMe-oF (NVMe over Fabrics) using RDMA over Ethernet (RoCE) or Fibre Channel is increasingly replacing SAS in hyperscale data centres. Major cloud providers (AWS, Azure, Google Cloud) and Indian hyperscalers (Reliance Jio, Airtel) deploy NVMe-oF storage for ultra-low-latency workloads as of 2024.

2.8 Memory Organization: RAM and ROM

Memory is the component that stores the instructions and data a computer currently needs. As introduced in Module 1, computer memory is divided into primary memory (directly accessible by the CPU) and secondary memory (storage). In this unit, we examine the detailed internal organisation, types, and characteristics of the primary memory components used in modern systems — specifically the RAM and ROM technologies that appear in every device from supercomputers to smartphones.

RAM — Random Access Memory

RAM (Random Access Memory) is volatile primary memory — it stores data only while power is supplied. The term 'random access' means that any memory location can be accessed in the same time regardless of its physical location (as opposed to sequential-access devices like magnetic tape). There are two fundamentally different RAM technologies:

1. SRAM — Static RAM

SRAM stores each bit using a flip-flop circuit — typically 6 transistors per cell. The data is held statically (as long as power is supplied) without any need for periodic refreshing. SRAM is extremely fast and is used wherever maximum speed is required, regardless of cost.

- Typical Access Time: 0.5–5 nanoseconds.
- Used for: CPU registers (sub-nanosecond), L1 cache (1–4 ns, 32–512 KB), L2 cache (3–10 ns, 256 KB–8 MB), L3 cache (10–40 ns, 8–96 MB).
- 2024–25 Example: Apple M3 Ultra (released March 2024) has 192 MB of on-package L3 cache (System-Level Cache, SLC) made from SRAM. Intel Core Ultra 200 series has up to 36 MB L3 cache; AMD Ryzen 9 9950X has 64 MB L3 cache (Zen 5, 2024).

- Cost: Very high (~100× more expensive per bit than DRAM). Not practical for main memory.

2. DRAM — Dynamic RAM

DRAM stores each bit using a single capacitor and one transistor (1T1C cell). The capacitor charge represents the bit value. Because capacitors leak charge, DRAM must be refreshed thousands of times per second to maintain data — this is the 'dynamic' aspect. DRAM is much denser and cheaper than SRAM, making it practical for gigabytes of main memory.

- Typical Access Time: 10–100 nanoseconds (without considering the memory controller and bus overhead).

Modern DRAM is always Synchronous DRAM (SDRAM) — operations are synchronised with the memory bus clock. The current generations:

Type	Max Speed	Bandwidth	Voltage	Status / Use (2024–25)
DDR4	DDR4-3200	~51 GB/s (dual-ch)	1.2 V	Still widely used; budget/mainstream desktops and laptops
DDR5	DDR5-6400 (std), up to DDR5-9600+ OC	~100 GB/s (dual-ch)	1.1 V	Standard in Intel Core Ultra 200, AMD Ryzen 7000/9000, 2024 systems
LPDDR5	LPDDR5-6400	~68 GB/s	0.5–1.05 V	Mobile/laptop DRAM; Qualcomm 8 Gen 3 smartphones, thin laptops
LPDDR5X	LPDDR5X-8533	~85 GB/s	0.5–1.05 V	Flagship 2024 smartphones (Snapdragon 8 Gen 3, Samsung Exynos 2400)
HBM3	Per-stack: 819 GB/s	3.2 TB/s (8-stack)	1.2 V	AI GPUs: NVIDIA H100/H200, AMD MI300X (2023-24); extremely high bandwidth
HBM3E	Per-stack: 1,228 GB/s	9.8 TB/s (8-stack)	1.1 V	NVIDIA B200/GB200 Blackwell (2024-25), AMD MI325X; AI training clusters

ROM — Read-Only Memory

ROM (Read-Only Memory) is non-volatile memory — it retains data without power. Originally, ROM was truly read-only after manufacture (Mask ROM). Modern systems use electrically reprogrammable variants:

- Mask ROM: Programmed during chip manufacture. Used for fixed microcode in simple embedded controllers. Cannot be updated.
- PROM (Programmable ROM): Programmed once by the user using a PROM programmer (fuse-blowing). Cannot be erased.
- EPROM (Erasable PROM): Erased by UV light exposure through a quartz window. Largely obsolete.
- EEPROM (Electrically Erasable PROM): Can be erased and reprogrammed electrically, byte by byte. Slow for large data; used for storing configuration data, serial numbers, and small data in embedded systems (e.g., Arduino EEPROM, BIOS CMOS settings chip).
- Flash Memory: A type of EEPROM that erases and programs in larger blocks (pages/sectors) rather than individual bytes. Dominant non-volatile storage technology for everything from USB drives to SSDs to smartphone storage.

Flash Memory Types in 2024–25:

- NOR Flash: Byte-addressable, fast random read, used for code execution in embedded systems (microcontroller firmware, UEFI BIOS chips). Example: Winbond W25Q series NOR Flash used in motherboard BIOS chips.
- NAND Flash: Block-addressable, dense, slower random read but fast sequential read/write, used for SSDs, USB drives, SD cards, and smartphone eMMC/UFS storage.

- 3D NAND (V-NAND): Flash cells stacked vertically in layers, dramatically increasing density. Samsung V-NAND (up to 290 layers as of 2024), SK Hynix 238-layer NAND, Micron 232-layer NAND. More layers = more capacity per chip.
- QLC NAND (Quad-Level Cell): 4 bits per cell — very high density and low cost but lower write endurance and speed. Used in high-capacity consumer SSDs (4 TB, 8 TB). Examples: Samsung 870 QVO, Crucial BX500.
- TLC NAND (Triple-Level Cell): 3 bits per cell — balance of density and endurance. Dominant in mainstream SSDs (Samsung 970 EVO Plus, WD Blue SN580).
- SLC/MLC NAND: 1 or 2 bits per cell — highest endurance and speed, used in enterprise SSDs and as cache ('SLC cache') in consumer SSDs.

Did You Know?

The UEFI (Unified Extensible Firmware Interface) — the modern replacement for BIOS — is stored in a

NOR Flash chip on the motherboard (typically 32–128 MB). When you power on a computer, the CPU starts

executing code directly from this NOR Flash chip before any operating system is loaded. The UEFI firmware

initialises all hardware (RAM, PCIe, USB), runs POST (Power-On Self-Test), and then loads the OS bootloader.

In 2024–25 systems, UEFI also handles Secure Boot — verifying the digital signature of the bootloader

to prevent rootkit attacks. This is why even if malware infects your OS, it cannot survive a firmware reset.

2.9 Memory Hierarchy and Speed-Cost Trade-offs

If we could build a computer with only one type of memory, the ideal choice would be memory that is infinitely fast, infinitely large, non-volatile, and free. Unfortunately, no such technology exists. The fundamental law of memory design is: faster memory is more expensive per bit and therefore smaller in capacity. The memory hierarchy is the elegant engineering solution to this constraint — a layered system that uses the right type of memory for each job, creating the illusion of a large, fast, affordable memory system.

The Principle of Locality of Reference

The memory hierarchy works because of a fundamental property of nearly all programs: they exhibit locality of reference — they tend to access the same data and instructions repeatedly, and tend to access data that is near (in address space) to recently accessed data.

- Temporal Locality: If a memory location is accessed now, it is likely to be accessed again soon. Example: a loop counter is accessed at every iteration of a loop — it is accessed repeatedly over a short time period.
- Spatial Locality: If a memory location is accessed, nearby locations are likely to be accessed soon. Example: accessing array[i] makes it likely that array[i+1] will be accessed next — they are at adjacent memory addresses.

These two properties mean that by keeping recently used data (temporal locality) and data near recently used addresses (spatial locality) in fast, small memory (cache), the CPU can find the data it needs quickly most of the time — without having to access the large, slow main memory.

The Memory Hierarchy — Levels

Modern systems organise memory in a hierarchy of five to six levels, with each level larger, slower, and cheaper per bit than the one above it:

Level	Technology	Typical Capacity	Access Time (2024-25)	Approx. Cost/GB
Registers	SRAM (in CPU die)	32–512 bytes	< 1 ns (1 cycle)	N/A — part of CPU chip
L1 Cache	SRAM (on-die)	32–128 KB per core	1–4 cycles	Very high (SRAM)
L2 Cache	SRAM (on-die)	256 KB – 4 MB/core	5–15 cycles	High (SRAM)
L3 Cache	SRAM (on-die/package)	8–96 MB shared	20–50 cycles	Moderate-high
Main Memory	DDR5 DRAM	16 GB – 256 GB	60–100 ns (~200 cycles)	Rs. 350–400/GB (2024)
NVMe SSD Storage	NAND Flash (PCIe 5.0)	500 GB – 4 TB	20–100 microseconds	Rs. 7–12/GB (2024)
HDD Storage	Magnetic disk	1 TB – 20 TB	3–15 milliseconds	Rs. 2–4/GB (2024)
Optical / Tape	Optical/Magnetic tape	100s of GB – PB range	Seconds to minutes	< Re. 0.50/GB (tape)

Source: Component pricing based on Indian market data (Amazon.in, Flipkart) and global component pricing (Newegg, B&H), Q3–Q4 2024.

How the Hierarchy Works in Practice

When the CPU needs data, it searches the hierarchy from the top (fastest) down until it finds the data:

- Cache Hit: The data is found in cache. The access is served in nanoseconds. No main memory access needed.
- Cache Miss: The data is not in cache. The CPU fetches the data from the next level (L2 → L3 → DRAM). A cache line (typically 64 bytes) is loaded into cache, capitalising on spatial locality.
- Page Fault: The data is not in RAM — it is on the SSD/HDD (virtual memory). The OS must load the page from storage — this takes microseconds to milliseconds and is very expensive. Programs that cause frequent page faults thrash — spending more time loading memory than computing.

Key Concept

Hit Rate and Miss Penalty:

HIT RATE: Fraction of accesses found in cache. Modern CPUs achieve L1 hit rates of 90–99%.

MISS PENALTY: The time cost of a cache miss — waiting for data from the next level.

Effective Access Time = Hit Rate × Cache Time + (1 - Hit Rate) × Miss Penalty

Example: L1 hit rate = 0.95, L1 access = 2 ns, DRAM access = 80 ns

EAT = 0.95 × 2 + 0.05 × 80 = 1.9 + 4.0 = 5.9 ns

Without cache (all DRAM): EAT = 80 ns — 13.5x SLOWER.

✓ Let's Check 2	
Q.No	**Question**
1	MCQ: Which of the following best describes the "cycle-stealing" mode of DMA operation? a) The DMA controller takes control of the entire bus for the complete transfer duration. b) The DMA controller steals one bus cycle at a time from the CPU, allowing the CPU to continue using the bus between transfers. c) The DMA controller operates only when the CPU is powered down. d) The CPU cycles between executing instructions and executing DMA transfers alternately.
2	MCQ: PCIe 5.0 with an x4 link configuration (as used by modern NVMe SSDs) provides what approximate bandwidth in one direction? a) ~4 GB/s b) ~8 GB/s c) ~16 GB/s d) ~32 GB/s
3	Fill in the Blank: The principle that a program tends to access the same memory locations repeatedly in a short time period is called _____ locality.
4	Fill in the Blank: In DRAM, each bit is stored as a charge on a _____, which must be periodically refreshed to prevent data loss.

5	True/False: NOR Flash memory is byte-addressable and allows code to be executed directly from the flash chip, making it suitable for storing UEFI/BIOS firmware.
6	True/False: A higher cache hit rate always increases effective memory access time, because the CPU spends more time checking the cache.
Answers	
1	b) The DMA controller steals one bus cycle at a time from the CPU — allowing the CPU to continue executing from cache between stolen cycles. This is slower than burst mode but less disruptive to CPU operation.
2	c) ~16 GB/s. PCIe 5.0 = ~4 GB/s per lane per direction. x4 = 4 × 4 = 16 GB/s per direction. This is why PCIe 5.0 NVMe SSDs (Crucial T705, Corsair MP700) achieve ~14,500 MB/s reads.
3	Temporal. Temporal locality = same location accessed repeatedly. Spatial locality = nearby addresses accessed in sequence.
4	Capacitor (one capacitor + one transistor = 1T1C cell). The capacitor leaks charge over time, requiring refresh cycles every ~64 ms.
5	True — NOR Flash supports random-byte read access (execute-in-place / XIP), making it ideal for code storage. Motherboard UEFI firmware is stored in a NOR Flash chip and executed directly without copying to RAM first.
6	False — A HIGHER cache hit rate REDUCES effective access time. More hits means more accesses are served by the fast cache at low latency, and fewer accesses require slow main memory. EAT = Hit_rate × T_cache + (1-Hit_rate) × T_memory.

Case Study

How India's Digital Public Infrastructure Scaled with Modern I/O and Memory Architecture: UPI at 14 Billion Transactions/Month (2024)

Background:

Unified Payments Interface (UPI) — developed by the National Payments Corporation of India (NPCI) and launched in April 2016 — has become the world's largest real-time payments system by volume. As of October 2024, UPI processed approximately 16.58 billion transactions worth Rs. 23.49 lakh crore in a single month (NPCI data, October 2024). This represents approximately 5,500 transactions per second at peak load — a remarkable feat of technology infrastructure that directly depends on the architectural concepts studied in this module.

The Technology Stack — Module 2 Concepts in Action:

1. Interrupt-Driven I/O and DMA in Payment Server Hardware:

Every UPI transaction involves a series of API calls between the payer's bank, NPCI's routing infrastructure, and the payee's bank — all within a 30-second SLA (Service Level Agreement) window. The servers running this infrastructure (deployed by NPCI's technology partners, primarily TCS and Infosys) use interrupt-driven I/O for network event handling — network interface cards (NICs) running at 25 Gbps or 100 Gbps raise interrupts when a packet arrives, triggering the payment processing thread. DMA with scatter-gather lists is used to move packet data from NIC receive buffers directly into application memory without CPU involvement.

2. PCIe and NVMe Storage for Transaction Logging:

Every UPI transaction must be durably logged — the system must be able to recover the exact state of every in-flight transaction in the event of a server failure. NPCI's data centres deploy enterprise NVMe SSDs (Samsung PM9A3 and Kioxia CM6 series, using PCIe 4.0 x4 with NVMe 1.4 protocol) for transaction logs. The fsync() write latency of these drives (the time to confirm a write is permanently stored) is under 100 microseconds — critical for meeting the 30-second SLA with margin to spare.

3. Memory Hierarchy for High-Frequency Transaction Processing:

The payment routing servers maintain in-memory databases (Redis clusters and custom in-house caches) that store frequently accessed data: VPA (Virtual Payment Address) resolution maps, bank routing tables, and recent transaction state. This data lives in DDR5 DRAM (64–512 GB per server), capitalising on temporal locality — the same VPA-to-bank mappings are looked up millions of times per hour. Hot transaction state is kept in L3 cache (32–64 MB on Intel Xeon Scalable 4th Gen / Sapphire Rapids processors deployed in major Indian data centres), enabling sub-microsecond lookup for the most active VPAs.

4. Addressing Modes in Payment Application Code:

The C++ and Java code running on NPCI servers makes heavy use of indexed and base+displacement addressing modes (compiled from array and struct accesses) for traversing transaction queues, linked lists of pending operations, and ring buffers for NIC packet handling. The efficiency of these addressing modes at the hardware level directly affects how many transactions per second can be processed on a given server.

Problem Solved:

In 2021, when UPI volume crossed 3 billion transactions/month, NPCI upgraded its infrastructure from PCIe 3.0 to PCIe 4.0 NVMe SSDs and from DDR4 to DDR5 DRAM — doubling transaction log write bandwidth and reducing 99th-percentile transaction latency by approximately 35%. This directly enabled the system to scale to the current 16+ billion transactions/month without a proportional increase in server count.

Learning Points:

1. DMA + interrupt-driven I/O together enable the high-throughput, low-latency packet processing that makes real-time payment systems possible.
2. The memory hierarchy principle — keeping hot data in DRAM and very hot data in cache — is the core architectural principle enabling sub-millisecond transaction response times.
3. Interface standards (PCIe generation upgrades) have measurable, quantitative impact on real-world system performance.
4. India's UPI infrastructure is a world-class example of how computer architecture principles translate directly into national-scale digital infrastructure.

Sources: NPCI UPI Monthly Statistics Dashboard, October 2024 (npci.org.in); Intel Xeon Scalable 4th Gen (Sapphire Rapids) technical specifications; Samsung PM9A3 NVMe SSD data sheet.

Summary

Module 2 has taken you through the operational core of how a computer's CPU accesses data, communicates with devices, and organises its memory. These topics move from the abstract (how an instruction finds its operands) to the intensely practical (how UPI processes 16 billion transactions a month using the same principles you have studied).

You began with addressing modes — the eight fundamental methods by which instructions locate their operands. From the simplest immediate mode (data in the instruction) to the powerful indexed mode (base + register for array access) and relative mode (PC + offset for branches), each addressing mode is precisely matched to a common programming pattern. Assembly language and instruction encoding then showed you how the CPU decodes binary instructions, and why RISC architectures use fixed 32-bit encoding while CISC architectures use variable-length encoding.

Subroutines — the fundamental unit of code organisation — are implemented using the CALL/RETURN mechanism and the stack. The stack frame model enables arbitrary nesting of function calls, parameter passing, and local variable storage, all without the calling code needing to know anything about the internal workings of the callee. Every function call in your Python or Java programs ultimately uses exactly this mechanism.

The I/O section showed you three progressively more sophisticated techniques: programmed I/O (polling) — simple but CPU-expensive; interrupt-driven I/O — efficient, allowing the CPU to do other work while waiting for devices; and DMA — offloading bulk data transfers entirely to a dedicated controller, freeing the CPU for computation. These three techniques are not competing alternatives — all three coexist in a modern system, each used where it is most appropriate.

The standard I/O interfaces examined — PCIe 6.0/7.0, NVMe 2.0, USB4 Version 2.0, and SAS-4 — represent the cutting edge of system interconnect technology as of 2024–25. Understanding these standards grounds the abstract architectural principles in the real hardware world.

Key highlights from the module:

- Eight addressing modes: immediate, register, direct, indirect, register-indirect, indexed, base+displacement, relative (PC). EA calculation formulas for each.
- Instruction encoding: Fixed-length (RISC — ARM, RISC-V, 32-bit) vs. variable-length (CISC — x86-64, 1–15 bytes). RISC-V R-type format: funct7 + rs2 + rs1 + funct3 + rd + opcode = 32 bits.
- Subroutines: CALL = push return address + jump; RETURN = pop return address + jump. Stack frame contains return address, saved registers, local variables.
- I/O techniques: Programmed I/O (polling — CPU busy-waits), Interrupt-driven (CPU does other work, ISR services device), DMA (bulk transfer without CPU; burst/cycle-stealing/transparent modes).
- Interrupts: IRQ → save state → IVT lookup → execute ISR → RETI. Types: hardware, software, timer, exception/trap, maskable, NMI. Modern systems use APIC + MSI-X.
- PCIe: Serial point-to-point; PCIe 5.0 = 4 GB/s/lane; PCIe 6.0 (2022) = 8 GB/s/lane; PCIe 7.0 spec finalised 2025 = 16 GB/s/lane.
- USB4 Version 2.0 (2022) = 80 Gb/s; compatible with Thunderbolt 5. USB PD 3.1 = up to 240 W charging.
- NVMe 2.0: Up to 65,535 queues × 65,535 depth. PCIe 5.0 NVMe (e.g., Crucial T705) = 14,500 MB/s read (2024).

- RAM: SRAM (fast, expensive, 6T/bit) for cache; DRAM (1T1C, must refresh) for main memory; DDR5 standard in 2024–25 PCs; LPDDR5X for mobile; HBM3E for AI GPUs (NVIDIA B200).
- Flash: NAND (QLC 4-bit, TLC 3-bit, SLC 1-bit) for SSDs; NOR for UEFI firmware (byte-addressable, XIP).
- Memory hierarchy: Registers → L1 → L2 → L3 → DRAM → NVMe SSD → HDD. Driven by locality of reference. EAT = h × T_cache + (1-h) × T_memory.

Glossary

Term	Definition
Effective Address (EA)	The actual memory address at which the operand is located, computed by the CPU from the addressing mode and instruction fields during the decode/execute phase.
Addressing Mode	The method specified in a machine instruction that defines how the CPU computes or locates the address of the instruction's operand.
Interrupt Service Routine (ISR)	A special subroutine that the CPU automatically jumps to when a hardware or software interrupt occurs. The ISR services the device and returns control to the interrupted program using a RETI instruction.
Direct Memory Access (DMA)	A hardware mechanism allowing a dedicated DMA controller to transfer data directly between a device and main memory without requiring the CPU to execute each individual data transfer instruction.
PCIe (PCI Express)	The dominant high-speed serial point-to-point I/O bus standard in modern computers, connecting GPUs, NVMe SSDs, and other high-performance expansion devices. PCIe 6.0 (2022) delivers 8 GB/s per lane.
NVMe (Non-Volatile Memory Express)	A protocol for accessing flash-based storage (SSDs) over PCIe, designed to exploit the parallelism of flash memory with up to 65,535 I/O queues. Far superior to the older AHCI protocol designed for spinning hard disks.

Memory Hierarchy	A structured organisation of memory types in a computer system, arranged from fastest/smallest/most expensive (registers, cache) to slowest/largest/cheapest (disk, tape), designed to provide the best combination of speed and capacity.
Locality of Reference	The empirical property of most programs that they access a relatively small subset of their data repeatedly (temporal locality) and that accesses cluster in nearby address ranges (spatial locality). This property makes cache memory effective.
Stack Frame (Activation Record)	A region of the stack allocated for a single subroutine call, containing the return address, saved register values, local variables, and parameters for that invocation.
DRAM Refresh	The periodic operation required to restore the charge in DRAM capacitor cells, which leak over time. Modern DRAM chips must be refreshed approximately every 32–64 ms to prevent data loss.

Self Assessment — Questions

Section A: Multiple Choice Questions

1. In register-indirect addressing mode, the instruction contains a register number. The effective address is:

a) The value stored in that register.

b) The value of the Program Counter plus the register value.

c) The address of the register itself.

d) The value stored at the memory location whose address is in the instruction.

2. Which addressing mode is MOST efficient for accessing elements of an array, where successive elements are at increasing memory addresses?

a) Immediate addressing

b) Direct addressing

c) Indexed addressing

d) Relative addressing

3. In relative addressing mode, the instruction contains a displacement of +20. If the Program Counter (PC) currently holds 3000 (pointing to the next instruction), what is the effective address of the branch target?

a) 20

b) 3000

c) 3020

d) 2980

4. A RISC-V R-type 32-bit instruction encodes which of the following fields?

a) Opcode, one register address, one immediate value

b) Opcode, destination register (rd), two source registers (rs1, rs2), funct3, funct7

c) Opcode and a 24-bit memory address only

d) Opcode, one source register, and one 16-bit immediate

5. When a CALL instruction is executed by the CPU, which of the following correctly describes the sequence of events?

a) The CPU jumps to the subroutine, and the return address is saved in the accumulator.

b) The CPU saves the return address onto the stack, then jumps to the subroutine address.

c) The CPU saves all registers to memory, then jumps to the subroutine.

d) The CPU decrements the Program Counter and jumps to the subroutine.

6. What is the purpose of the Frame Pointer (FP / Base Pointer BP) register in subroutine calls?

a) It points to the top of the entire program's memory region.

b) It provides a stable reference point to the base of the current stack frame, enabling fixed-offset access to local variables even when SP changes.

c) It stores the return address for the current subroutine.

d) It counts the number of active subroutine calls currently on the stack.

7. In memory-mapped I/O, how does the CPU access an I/O device register?

a) By using dedicated IN and OUT assembly instructions with port numbers.

b) By sending a special I/O request signal on the control bus.

c) By executing ordinary LOAD and STORE instructions to the device's assigned memory address.

d) By calling a privileged BIOS function through a software interrupt.

8. In programmed I/O (polling), the CPU reads the device status register repeatedly until the device is ready. What is the PRIMARY disadvantage of this approach?

a) Polling is unreliable and may miss device signals.

b) The CPU wastes 100% of its time busy-waiting and cannot perform other work.

c) Polling requires special hardware support not available in all processors.

d) Polling can only be used with slow devices such as keyboards.

9. During interrupt handling, which CPU action occurs IMMEDIATELY after an interrupt request is detected (assuming interrupts are enabled)?

a) The CPU discards the current instruction and restarts execution from address 0.

b) The CPU finishes the current instruction, saves the processor state (PC and key registers), then jumps to the ISR.

c) The CPU writes a message to the device's control register.

d) The CPU resets the interrupt controller.

10. What is a Non-Maskable Interrupt (NMI)?

a) An interrupt that can be delayed by setting the interrupt flag to 0 in the status register.

b) An interrupt generated by a software INT instruction.

c) An interrupt that cannot be disabled or delayed, used for critical hardware failures such as power loss or memory errors.

d) An interrupt used by the operating system for system calls.

11. In DMA burst mode, the DMA controller:

a) Transfers one word per bus cycle, then releases the bus back to the CPU.

b) Takes control of the bus and transfers the entire data block in one continuous operation without releasing the bus.

c) Shares the bus equally with the CPU, alternating bus control every clock cycle.

d) Transfers data only when the CPU is in an idle state.

12. PCIe 6.0, released in 2022, provides what approximate bandwidth per lane per direction?

a) ~2 GB/s

b) ~4 GB/s

c) ~8 GB/s

d) ~16 GB/s

13. Which of the following correctly describes a key advantage of NVMe over the older AHCI (SATA) protocol for SSD access?

a) NVMe drives are physically larger, allowing more storage cells.

b) NVMe supports up to 65,535 I/O queues with up to 65,535 commands each, versus AHCI's single queue of 32 commands.

c) NVMe uses a serial interface while AHCI uses a parallel interface.

d) NVMe drives do not require a device driver.

14. USB4 Version 2.0, announced in 2022 and shipping in 2023–24 systems, provides a maximum bandwidth of:

a) 10 Gb/s

b) 40 Gb/s

c) 80 Gb/s

d) 120 Gb/s

15. Which RAM technology stores each bit as a charge in a capacitor, requires periodic refresh, and is used for main memory (RAM) in computers?

a) SRAM (Static RAM)

b) DRAM (Dynamic RAM)

c) FLASH Memory

d) EEPROM

16. DDR5 DRAM, which became the standard in high-performance desktop and laptop systems by 2024, offers which of the following improvements over DDR4?

a) Higher voltage (1.4 V vs. 1.2 V) for improved stability.

b) Higher bandwidth (up to ~100 GB/s dual-channel), lower voltage (1.1 V), and higher module capacity.

c) Smaller physical connector, reducing motherboard space.

d) Non-volatile storage, retaining data when power is lost.

17. What type of flash memory is used in UEFI/BIOS firmware chips on motherboards, and why?

a) NAND Flash, because it is very fast for sequential writes.

b) QLC NAND, because it has the highest storage density.

c) NOR Flash, because it is byte-addressable and supports execute-in-place (XIP) — allowing code to run directly from the flash chip.

d) HBM3, because it has the highest bandwidth.

18. The principle of spatial locality in the context of the memory hierarchy means:

a) Programs tend to access the same data repeatedly within a short time.

b) Programs tend to access memory locations that are physically close to recently accessed locations.

c) Frequently accessed data should be stored in geographically close data centres.

d) The memory bus should be physically short to reduce signal propagation delay.

19. If the L1 cache hit rate is 0.92, L1 cache access time is 3 ns, and DRAM access time (on miss) is 90 ns, what is the Effective Memory Access Time (EMAT)?

a) 93 ns

b) 10.0 ns

c) 7.92 ns

d) 45 ns

20. Which of the following I/O interface standards is primarily used in enterprise servers and data centres for connecting high-reliability hard drives and enterprise SSDs, and supports multi-path I/O for high availability?

a) USB 3.2 Gen 2

b) PCIe 4.0

c) SAS (Serial Attached SCSI)

d) Thunderbolt 4

Section B: True / False (5 Questions)

1. In indirect addressing mode, the instruction contains a memory address that itself holds the effective address (the address of the operand) — requiring two memory accesses to retrieve the operand.

2. PCIe uses a shared parallel bus architecture, meaning all devices connected to the PCIe bus must share the total available bandwidth.

3. SRAM retains its data as long as power is supplied and does NOT require periodic refresh cycles, making it faster but more expensive per bit than DRAM.

4. The IOMMU (Input-Output Memory Management Unit) provides memory address translation and access control for DMA operations, preventing unauthorised devices from accessing arbitrary physical memory.

5. In a memory hierarchy system, a "page fault" occurs when the CPU attempts to access data that is present in the L1 cache but not in the L2 cache.

Section C: Short Answer Questions (5 Questions)

1. Explain the difference between indexed addressing mode and base+displacement addressing mode. In what programming contexts is each mode most commonly used? Provide a concrete example (with hypothetical register values and memory addresses) for each.

2. Describe the complete sequence of events from the moment an I/O device raises an interrupt request (IRQ) to the moment the interrupted program resumes execution. Name all registers, data structures, and hardware components involved.

3. Explain the three DMA transfer modes (burst, cycle-stealing, and transparent). For each mode, describe: (a) how the bus is managed, (b) the impact on CPU performance during the transfer, and (c) a scenario where that mode would be preferred.

4. Compare DDR5 DRAM and HBM3E memory in terms of bandwidth, capacity per package, power, cost, and application domain. Provide at least one real-world product example for each (2024–25 data).

5. A computer has an L1 cache with a hit rate of 0.90 and an access time of 2 ns, an L2 cache with a hit rate of 0.85 (when L1 misses) and an access time of 8 ns, and DRAM with an access time of 80 ns. Calculate the Effective Memory Access Time (EMAT) for a two-level cache hierarchy. Show all working clearly**.**

Self Assessment — Answer Key

Section A: MCQ

Q.No	Correct Answer / Explanation
1	a) The value stored in that register is the EA. In register-indirect mode, the register holds the address of the operand in memory. E.g., LOAD R1, (R2) — R2 contains the address; the operand is at Memory[R2].
2	c) Indexed addressing. The index register is incremented each iteration to advance through the array. EA = Base_address + Index_register allows A[0], A[1], A[2]... to be accessed by incrementing the index.
3	c) 3020. EA = PC + Displacement = 3000 + 20 = 3020. Relative addressing is used for all branch instructions in modern ISAs; the displacement is signed, so negative values implement backward branches (loops).
4	b) opcode, rd, rs1, rs2, funct3, funct7 — totalling 32 bits. RISC-V R-type: funct7(7) + rs2(5) + rs1(5) + funct3(3) + rd(5) + opcode(7) = 32 bits.
5	b) The CPU saves the return address (address of the instruction after CALL) onto the stack, then loads the PC with the subroutine's starting address, causing a jump. RETURN pops the saved address back into the PC.
6	b) The FP provides a stable base address for the current stack frame. Local variables are accessed at fixed offsets from FP even as SP changes during the subroutine (e.g., due to nested PUSH operations). Without FP, SP-relative offsets would change dynamically.
7	c) By executing ordinary LOAD and STORE instructions to the device's assigned memory address. Memory-mapped I/O maps device registers into the regular memory address space — no special instructions are needed.

8	b) The CPU wastes 100% of its time busy-waiting in the polling loop and cannot perform other work. Even if no I/O event occurs for seconds, the CPU is fully occupied checking the status register repeatedly.
9	b) Finish current instruction → save PC and registers to stack → look up ISR address in IVT → jump to ISR. The CPU does NOT discard work or reset; it saves its state precisely so the interrupted program can be resumed exactly.
10	c) NMI cannot be disabled by clearing the interrupt flag. Used for critical hardware events: power-loss detection, hardware memory error (ECC), watchdog timer expiry. The CPU must always respond to an NMI.
11	b) In burst mode, the DMAC seizes the bus and transfers the entire block continuously. Fastest for bulk transfers; completely blocks CPU memory access during the burst. Used for large block transfers where latency is acceptable.
12	c) ~8 GB/s per lane per direction. PCIe 6.0 doubles PCIe 5.0's 4 GB/s/lane, achieving 8 GB/s/lane. x16 slot = 8 × 16 = 128 GB/s per direction — targeting AI accelerators and 800G network infrastructure.
13	b) NVMe supports up to 65,535 queues × 65,535 commands per queue, enabling massive I/O parallelism. AHCI has one queue of 32 commands — designed for spinning HDDs that could only handle one I/O at a time. Flash SSDs can handle thousands of simultaneous I/Os.
14	c) 80 Gb/s. USB4 Version 2.0 doubles USB4 Gen 3x2's 40 Gb/s to 80 Gb/s, matching Thunderbolt 5. Thunderbolt 5 extends this to 120 Gb/s asymmetric for display/GPU bandwidth.
15	b) DRAM. Each DRAM cell = 1 capacitor + 1 transistor (1T1C). Capacitors leak, requiring refresh every ~64 ms. DRAM provides large capacity at low

	cost, making it practical for GBs of main memory. SRAM uses 6 transistors per cell, is faster but far more expensive.
16	b) DDR5 provides higher bandwidth (~100 GB/s dual-channel vs. ~51 GB/s for DDR4-3200), lower operating voltage (1.1 V vs. 1.2 V), higher maximum capacity per module (up to 128 GB RDIMM vs. 32 GB for DDR4), and on-die ECC.
17	c) NOR Flash, because it supports byte-level random read access and execute-in-place (XIP) — the CPU can fetch and execute instructions directly from NOR Flash without copying them to RAM. NAND Flash requires data to be read in pages and loaded into RAM before execution.
18	b) Spatial locality — accessing memory locations near recently accessed locations. Array traversal (A[0], A[1], A[2]...) is the canonical example. Cache lines load 64 bytes at a time, pre-loading neighbouring array elements and exploiting spatial locality.
19	c) EMAT = (0.92 × 3) + (0.08 × 90) = 2.76 + 7.2 = 9.96 ns ≈ 10.0 ns. Without cache: 90 ns. Cache improves effective access by 9x. Formula: EMAT = h × T_cache + (1 - h) × T_DRAM.
20	c) SAS (Serial Attached SCSI). SAS supports dual-path I/O (two independent paths to each drive — critical for fault tolerance in servers), full-duplex operation, and is the standard in enterprise HPE, Dell, and IBM server storage. NVMe is faster but SAS remains dominant in enterprise for reliability and dual-path support.

Section B: True / False Answer Key

Q.No	Answer / Explanation
1	True — Indirect addressing requires TWO memory accesses: (1) fetch the pointer (address of operand) from the address in the instruction, (2) fetch the actual operand from the address read in step 1. This is why indirect addressing is slower than direct or register-indirect modes.
2	False — PCIe is a POINT-TO-POINT serial interface, NOT a shared bus. Each PCIe device has a dedicated link (one or more lanes) to the CPU or chipset. Devices do not share bandwidth with each other. The older PCI was a shared parallel bus; PCIe replaced it precisely to eliminate bus contention.
3	True — SRAM uses 6-transistor flip-flop cells that hold their state as long as power is applied. No refresh is needed. This makes SRAM significantly faster than DRAM, but the 6T cell is much larger and more expensive per bit than DRAM's 1T1C cell.
4	True — The IOMMU (Intel VT-d / AMD-Vi) maps device-visible I/O virtual addresses to physical memory addresses and enforces access controls. Without an IOMMU, a malicious or compromised DMA-capable device could read or corrupt any physical memory location — a DMA attack. The IOMMU prevents this and is also essential for VM device assignment.
5	False — A page fault occurs when the CPU attempts to access a virtual address whose corresponding page is NOT present in MAIN MEMORY (RAM) — it is on the secondary storage (SSD/HDD). The OS must then load the page from storage. A miss between L1 and L2 cache is simply called a cache miss, not a page fault.

Section C: Short Answer — Suggested Answers

Q.No	Suggested Answer
1	Indexed: EA = Base_constant (in instruction) + Index_register. Used for array access. Example: if array A starts at 1000 and element size is 4 bytes, then A[3] is at 1000 + 3*4 = 1012. Instruction: LOAD R1, 1000(R4) where R4 = 12. EA = 1000 + 12 = 1012. Base+Displacement: EA = Base_register + Displacement_constant. Used for accessing struct fields or stack frame variables. Example: if struct starts at R5 = 2000 and field "salary" is at offset 8, then: LOAD R2, 8(R5) → EA = 2000 + 8 = 2008. Key distinction: in indexed mode the base is a constant and the index varies; in base+displacement the base register varies (points to different struct instances) while the displacement is fixed (selects the same field).
2	Sequence: (1) Device raises IRQ signal on its interrupt request line. (2) Interrupt controller (APIC/I/O APIC) receives the IRQ and forwards an interrupt number (vector) to the CPU if the interrupt priority is high enough. (3) CPU finishes its current instruction. (4) CPU checks the Interrupt Enable Flag (IF) in the status register — if IF=1, interrupt is accepted. (5) CPU pushes current PC (return address) and FLAGS register onto the stack. (6) CPU may push additional registers depending on architecture. (7) CPU uses the interrupt vector number to index into the Interrupt Descriptor Table (IDT/IVT) and reads the ISR address. (8) PC is loaded with ISR address — execution jumps to ISR. (9) ISR executes: reads data from device, clears the device's interrupt flag, performs necessary processing. (10) ISR executes RETI (Return from Interrupt). (11) CPU pops FLAGS and PC from stack — original program resumes from exactly where it was interrupted.

3	Burst Mode: DMAC seizes the bus and transfers the complete block without releasing it. CPU cannot access memory during the transfer (can continue if working from cache). Fastest transfer rate. Used for: bulk data loads from disk where maximum throughput is needed and brief CPU memory stalls are acceptable. Cycle-Stealing: DMAC steals one bus cycle, transfers one word, releases bus, repeats. CPU can interleave memory accesses between stolen cycles. Moderate transfer speed. Used for: audio/video streaming where data must flow continuously but CPU must also process the stream in real time. Transparent Mode: DMAC only uses the bus when CPU is not using it (CPU executing ALU operations from registers). Slowest but completely invisible to CPU. Used for: background data prefetch or low-priority transfers in simple embedded systems where CPU performance must not be compromised at all.
4	DDR5 DRAM (2024-25): Bandwidth ~100 GB/s dual-channel (DDR5-6400); Capacity up to 128 GB per DIMM; Voltage 1.1 V; Cost ~Rs. 350–400/GB (Indian market Q4 2024); Application: CPU main memory in desktops, laptops, servers. Example: 32 GB DDR5-6000 kit used with AMD Ryzen 9 9950X or Intel Core Ultra 9 285K. HBM3E (2024-25): Bandwidth 1,228 GB/s per stack, ~9.8 TB/s for 8-stack package; Capacity 24–36 GB per package (stacked on GPU die); Voltage 1.1 V; Cost: not sold separately — integrated into GPU packages; Application: AI/ML training accelerators. Example: NVIDIA H200 GPU (80 GB HBM3E, 4.8 TB/s, released Q1 2024); AMD MI325X (256 GB HBM3E, 6.0 TB/s, 2024). HBM3E provides ~50x more bandwidth than DDR5 but is far smaller in capacity and integrated directly into the processor package.
5	Two-level cache EMAT formula: EMAT = h1 × T_L1 + (1-h1) × [h2 × T_L2 + (1-h2) × T_DRAM]. Given: h1=0.90, T_L1=2 ns; h2=0.85, T_L2=8 ns; T_DRAM=80 ns. Step 1: L2 hit or DRAM miss cost (when L1 misses): = h2 ×

	T_L2 + (1-h2) × T_DRAM = 0.85 × 8 + 0.15 × 80 = 6.8 + 12.0 = 18.8 ns. Step 2: Overall EMAT: = h1 × T_L1 + (1-h1) × 18.8 = 0.90 × 2 + 0.10 × 18.8 = 1.8 + 1.88 = 3.68 ns. Interpretation: With a two-level cache, the effective access time is 3.68 ns — vs. 80 ns for pure DRAM access (21.7x speedup). This illustrates why modern CPUs invest heavily in large, multi-level cache hierarchies.

✓ Let's Check 3

Q.No	Question
1	MCQ: A program accesses memory locations 5000, 5001, 5002, 5003 in sequence. This is an example of which type of locality? a) Temporal locality b) Spatial locality c) Address locality d) Sequential locality
2	MCQ: Which of the following NVMe SSDs, available in 2024, achieves the highest sequential read speed by using a PCIe 5.0 x4 interface? a) Samsung 990 Pro (PCIe 4.0) b) WD Black SN850X (PCIe 4.0) c) Crucial T705 (PCIe 5.0) d) Samsung 870 EVO (SATA)
3	Fill in the Blank: The DMA technique where the DMAC only uses the memory bus when the CPU is not using it is called _____ mode.
4	Fill in the Blank: In x86-64 systems, the modern interrupt architecture that replaced the legacy 8259A PIC and supports multi-core interrupt routing is called _____ (APIC).

5	True/False: HBM3E memory (as used in NVIDIA H200 and AMD MI325X GPUs in 2024) provides higher bandwidth than DDR5 DRAM but much smaller total capacity.
6	True/False: SAS (Serial Attached SCSI) supports multi-path I/O, meaning a single drive can be connected to two separate controllers simultaneously for redundancy — a feature not available in standard NVMe.
Answers	
1	b) Spatial locality — the program accesses memory locations that are physically adjacent (sequential addresses 5000, 5001, 5002, 5003). Temporal locality would mean accessing the SAME location repeatedly. Cache lines (64 bytes) exploit spatial locality by loading neighbouring bytes together.
2	c) Crucial T705 (PCIe 5.0) — achieves up to 14,500 MB/s sequential read (PCIe 5.0 x4). The Samsung 990 Pro and WD Black SN850X use PCIe 4.0 x4 (~7,300–7,450 MB/s). Samsung 870 EVO is a SATA SSD (~560 MB/s).
3	Transparent (Background). In transparent mode, the DMAC transfers only during bus idle cycles when the CPU is executing internal operations. Slowest but fully transparent to CPU performance.
4	Advanced Programmable Interrupt Controller (APIC). Consists of a Local APIC per core and one or more I/O APICs. Modern devices use MSI/MSI-X to signal interrupts directly without the I/O APIC.
5	True — HBM3E (e.g., NVIDIA H200: 4.8 TB/s bandwidth, 80 GB capacity) provides vastly higher bandwidth than DDR5 (~100 GB/s dual-channel) but much less capacity (80–256 GB per GPU package vs. many TB of

	DDR5 in server systems). HBM3E trades capacity for bandwidth by stacking DRAM dies directly on the processor package.
6	True — SAS supports dual-port drives: a single SAS drive has two ports that can connect to two independent controllers. If one controller or cable fails, the drive remains accessible through the second path — critical for enterprise high-availability storage. Standard NVMe does not natively support multi-path in the same way (NVMe multipath requires NVMe-oF or specific enterprise configurations).

JGI JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA